Edit Distance (LeetCode 72)

Comprehensive Analysis & Implementation Guide

Problem Statement

Given two strings `word1` and `word2`, return the minimum number of operations required to convert `word1` into `word2`.

You are allowed to perform only three operations:

- Insert a character
- Delete a character
- Replace a character

Example

Input:

`word1 = "horse"`

`word2 = "ros"`

Output:

`3`

Explanation:

```
horse
↓
rorse         (Replace 'h' with 'r')
↓
rose          (Delete 'r')
↓
ros           (Delete 'e')
```

Minimum operations = 3

Understanding the Problem

The question is not asking us to print the operations.

It only asks:

> ***"What is the minimum number of operations required?"***

At every mismatch between two characters, we have three possible choices:

- Replace
- Delete
- Insert

Our goal is to choose the operation that eventually gives the minimum cost.

DP State

Let **dp[i][j]** represent:

> Minimum operations required to convert `word1[0...i]` into `word2[0...j]`

Notice that we are always solving the problem for prefixes of both strings.

Recursive Intuition

Suppose:

`word1 = horse`

`word2 = ros`

Current comparison

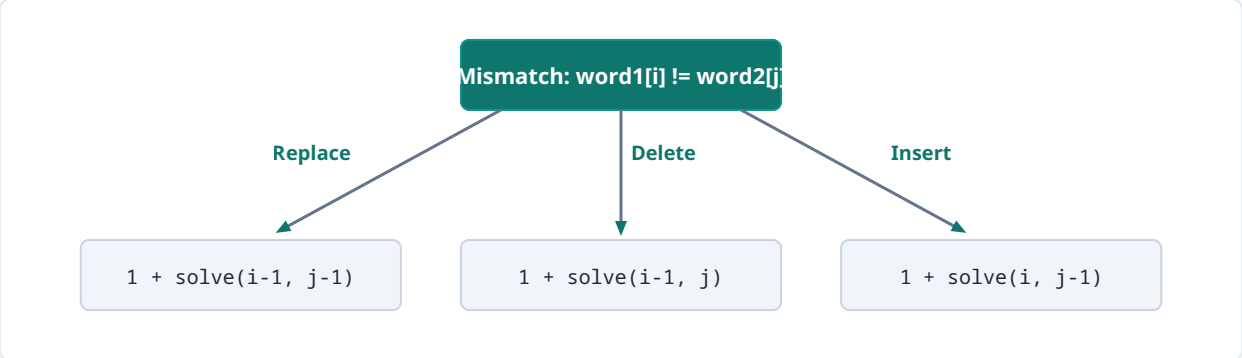
```
horse
    ↑
    e

ros
  ↑
  s
```

i = 4, **j = 2**

Characters: **e != s**

Now we have exactly three choices.



Case 1 : Replace

Imagine replacing:

e → s

The strings become:

```
horss
ros
```

Now the last characters already match:

```
hors(s)
ros (s)
```

Since they are already equal, we don't need to compare them again. We simply solve the remaining prefixes:

```
hors
ro
```

Hence:

Replace ↓ 1 + solve(i-1,j-1)

Case 2 : Delete

Delete the current character from word1.

horse ↓ hors

Now compare:

```
hors
ros
```

Only word1 became shorter.

Therefore:

Delete ↓ 1 + solve(i-1,j)

Case 3 : Insert

Insert the current character of word2.

horse ↓ horses

The inserted character already matches the current character of word2. So we don't compare it again. Only word2 moves backward.

Therefore:

Insert ↓ 1 + solve(i,j-1)

Transition

If characters match:

```
word1[i] == word2[j]
```

No operation is needed.

```
dp[i][j] = dp[i-1][j-1]
```

Otherwise:

```
dp[i][j] = 1 + min(Replace, Delete, Insert)
```

or

```
dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
```

Biggest Doubt

"The recursion never actually replaces the character."

Correct. It never changes the string. This is intentional.

Why? Dynamic Programming is not simulating the edits. It is calculating the minimum cost assuming that a particular operation has already been performed.

Example:

Current:

```
horse
    ↑ e
ros
  ↑ s
```

Suppose we choose Replace. We imagine:

```
e → s
```

Now:

```
horss
ros
```

Since these two characters are now equal, they are finished. The remaining problem becomes:

```
hors
ro
```

Therefore the recursion moves to `(i-1,j-1)`.

Notice that we never actually write `word1[i]=word2[j];` because the recursion only cares about the remaining prefixes. The operation is conceptual, not physical.

Think Like This:

Imagine someone asks: "If I replace this character, what will be the minimum cost afterwards?" Instead of modifying the string, the recursion directly jumps to solving the remaining subproblem. This is why Dynamic Programming works efficiently.

Base Cases

These are another common source of confusion.

Base Case 1

```
if(i<0) return j+1;
```

What does this mean?

```
word1 = ""
word2 = word2[0...j]
```

Example:

```
word1 = ""
word2 = "ros"
```

How do we convert `""` → `"ros"`? Only one operation is possible: Insert every remaining character.

- Insert r
- Insert o
- Insert s

Total operations = 3

Indices: **r(0) o(1) s(2)** → Last index **j = 2**

Number of characters = **2 + 1 = 3**

Therefore: **return j+1**

Base Case 2

```
if(j<0) return i+1;
```

Meaning:

```
word1 = word1[0...i]
word2 = ""
```

Example:

```
horse → ""
```

Only operation possible: Delete every character.

- Delete h
- Delete o
- Delete r
- Delete s
- Delete e

Total = 5

Indices: **h(0) o(1) r(2) s(3) e(4)** → Last index = 4

Characters = **4 + 1 = 5**

Therefore: **return i+1**

Why +1?

Indices begin from 0.

For string `abc`:

- Indices: **a(0) b(1) c(2)**
- Last index = 2
- Characters = 3, which equals **2 + 1**

Hence: **return i+1** or **return j+1**

Complete Dry Run

Input:

`word1 = horse`

`word2 = ros`

Initially:

```
horse
    ↑
ros
  ↑
```

e != s

Three choices:

- Replace ↓ 1 + solve("hors","ro")
- Delete ↓ 1 + solve("hors","ros")
- Insert ↓ 1 + solve("horse","ro")

The recursion computes all three possibilities.

After memoization, suppose the costs become:

```
Replace = 3, Delete = 2, Insert = 4
```

Then:

1 + min(3,2,4) = 3

So deleting gives the optimal answer at this stage. Every recursive call follows the same logic until one string becomes empty.

Memoization Table

Rows = `word1`, Columns = `word2`

	r	o	s
h			
o			
r			
s			
e			

Each cell stores:

> *Minimum operations required to convert word1[0...i] into word2[0...j]*

Once computed, the value is reused, avoiding repeated recursion.

Tabulation Approach (Bottom-Up DP)

Now let's understand how we solve this problem using tabulation (bottom-up DP). Instead of recursion, we build the answer from smaller subproblems.

Step 1: Table Definition

We create a table: `dp[n+1][m+1]`

Where:

`n = length of word1`
`m = length of word2`

Here, `dp[i][j]` means:

> Minimum operations to convert first i characters of word1 into first j characters of word2

Step 2: Initialize Base Cases

First Row (i = 0): Convert empty string to word2:

```
dp[0][j] = j   (Insert all characters)
```

First Column (j = 0): Convert word1 to empty string:

```
dp[i][0] = i   (Delete all characters)
```

Step 3: Fill the Table

We fill the table row by row.

If characters match:

`dp[i][j] = dp[i-1][j-1]`

Else:

```
dp[i][j] = 1 + min(
    dp[i-1][j-1],   // Replace
    dp[i-1][j],     // Delete
    dp[i][j-1]      // Insert
)
```

Step 4: Complete Table for Example

`word1 = horse`, `word2 = ros`

We build the table:

	""	r	o	s
""	**0**	1	2	3
h	1	1	2	3
o	2	2	**1**	2
r	3	2	2	2
s	4	3	3	2
e	5	4	4	**3**

Step 5: Understanding Table Filling

Let's see a few important cells:

- **Cell (1,1)** → comparing 'h' and 'r': **dp[1][1] = 1 + min(0,1,1) = 1**
- **Cell (2,2)** → comparing 'o' and 'o' (Characters match): **dp[2][2] = dp[1][1] = 1**
- **Cell (5,3)** → comparing 'e' and 's': **dp[5][3] = 1 + min(2,2,3) = 3**

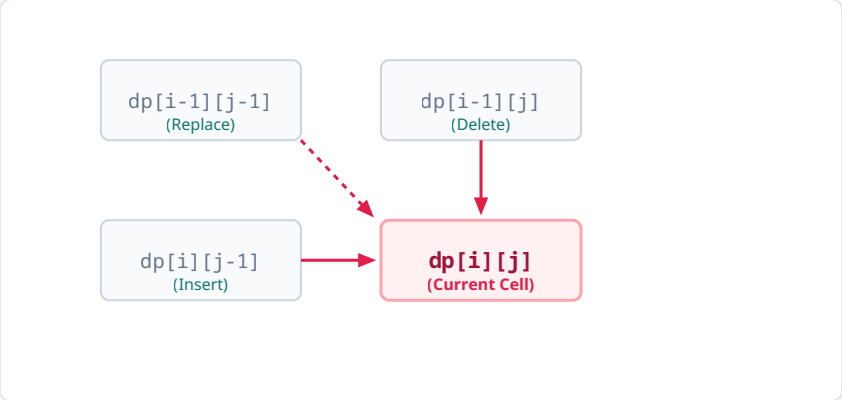
Final Answer

dp[n][m] = dp[5][3] = 3

Key Idea of Tabulation

We start from smallest problems (empty strings) and gradually build up to full strings. Each cell depends only on:

- Left
- Top
- Diagonal



Algorithm

1. Define the state as the minimum operations to convert `word1[0...i]` into `word2[0...j]`.
2. If both current characters are equal, move diagonally without any cost.
3. Otherwise, try all three operations:
   - Replace → **(i-1, j-1)**
   - Delete → **(i-1, j)**
   - Insert → **(i, j-1)**
4. Add one operation to the chosen move.
5. Store the answer in the DP table.
6. Return the stored value whenever the same state is encountered again.

Complexity

Time Complexity: **O(N × M)** — Every state is solved only once.

Space Complexity: **O(N × M)** — for the DP table.

Key Interview Takeaways

- The DP state always represents prefixes of both strings.
- The algorithm never physically edits the strings.

- Each recursive call assumes the chosen operation has already been performed and solves only the remaining subproblem.
- Replace moves diagonally because both characters become equal after the conceptual replacement.
- Delete moves up because only word1 becomes shorter.
- Insert moves left because the inserted character already matches the current character of word2.
- If one string becomes empty:
  - Insert all remaining characters of the other string (`j + 1` operations), or
  - Delete all remaining characters of the current string (`i + 1` operations).

These ideas form the foundation of the Edit Distance DP solution and are frequently asked in coding interviews.